Credit Card Approval Prediction: ML-Powered Applicant Risk Assessment

Project Description:

Credit Card Approval Prediction is a machine-learning-powered web application that estimates the likelihood a credit card applicant will be approved, based on financial, demographic, and employment information collected on a standard application form. The system replaces slow, inconsistent manual review with a real-time, data-driven risk assessment that a bank credit analyst can use as one input to a lending decision.

The application is built around a tuned XGBoost classifier trained on historical applicant and credit-record data. Users submit an applicant's details (age, income, employment status, education, family situation, housing type, and more) through a Flask-based web form. The backend converts these inputs into the exact one-hot-encoded feature vector the model expects, runs inference, and returns an Approve/Reject decision alongside the underlying approval probability, all within seconds.

A key design choice is transparency about model limitations: because rejections are rare in the training data (roughly 1.7% of applicants), the classification threshold was tuned via cross-validation (0.29, not the default 0.5) to better balance precision and recall on the minority “risky applicant” class. The result page discloses the model's actual precision and recall for the reject class so the prediction is never presented as more reliable than it is.

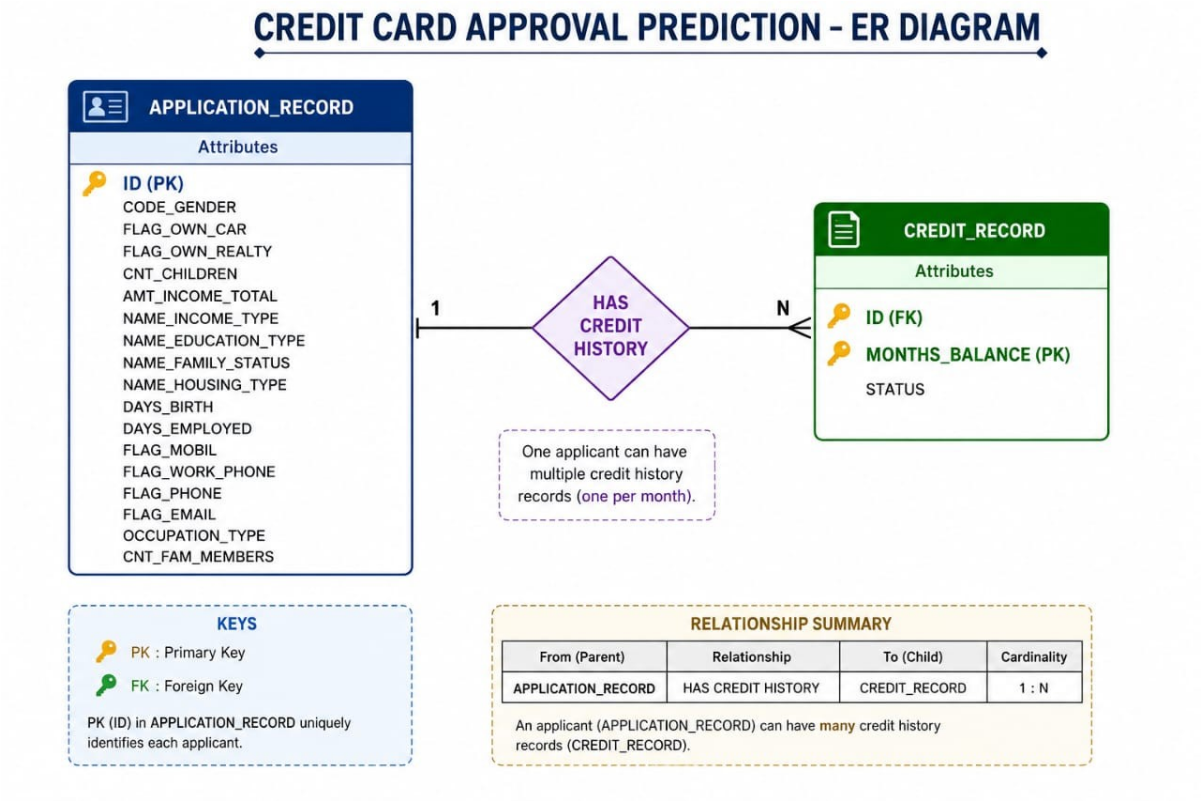
Scenarios

Scenario 1: A bank credit analyst enters details for a salaried applicant — age 34, Working income type, Higher education, owns realty, 6 years employed, no dependents. The system converts these into a feature vector, runs it through the tuned XGBoost model, and returns an “Approve” decision with the approval probability displayed on a gauge.

Scenario 2: An applicant with a thin credit history and a high dependents ratio (multiple children, currently unemployed, renting) is submitted. The model returns its decision along with the approval probability, and the result page's disclosure panel reminds the analyst that this is one input to a decision, not an automated final verdict.

Scenario 3: A prospective applicant uses the public-facing form themselves to self-check eligibility before formally applying at the bank, entering approximate income, housing type (rented apartment), and occupation (IT staff). The gauge visualization on the result page shows the approval probability at a glance, helping them decide whether to apply now or first improve their financial profile.

Technical Architecture:



Entity Relationship Diagram — APPLICATION_RECORD to CREDIT_RECORD (1:N)

Credit Card Approval Prediction uses a modular pipeline architecture spanning data preprocessing, model training, and a lightweight Flask serving layer. Raw applicant and credit-record data flows from the source dataset, through cleaning and feature engineering (Epic 3), into a cross-validated model-training and threshold-tuning stage (Epic 4). The trained XGBoost model and its metadata (feature columns, decision threshold, class labels) are loaded once at application startup and served through two Flask routes: a form-rendering home page and a /predict endpoint that performs inference and renders the result page.

(Note: This project uses a database-style relationship between applicant and credit-history records, so the ER diagram above is included alongside its description, as required.)

Pre-requisites:

- Python Programming Proficiency: Python 3.9+
- Flask Framework Knowledge: Flask web framework fundamentals
- Machine Learning Libraries: scikit-learn, XGBoost, pandas, NumPy, joblib
- HTML, CSS, and JavaScript Skills: for the applicant form and result page
- Version Control with Git: Git / GitHub
- Development Environment Setup: virtualenv / venv, pip

Project Workflow:

Milestone 1: Data Understanding, Cleaning & Feature Engineering

- Activity 1.1: Source and explore the application_record.csv and credit_record.csv datasets.
- Activity 1.2: Research and select the appropriate ML model for tabular credit-risk classification.
- Activity 1.3: Define the end-to-end application architecture.
- Activity 1.4: Set up the development environment and project structure.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop feature engineering and label-derivation logic.
- Activity 2.2: Implement the Flask backend routing and request handling.

Milestone 3: App.py Development

- Activity 3.1: Write the main application logic in app.py, establishing routes and model inference.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the applicant form and result page UI.
- Activity 4.2: Create dynamic result rendering (probability gauge, disclosure panel) using Jinja templates.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the environment and dependencies.
- Activity 5.2: Run local testing and verification across multiple applicant scenarios.
- Activity 5.3: Verify predictions end-to-end and document known limitations.

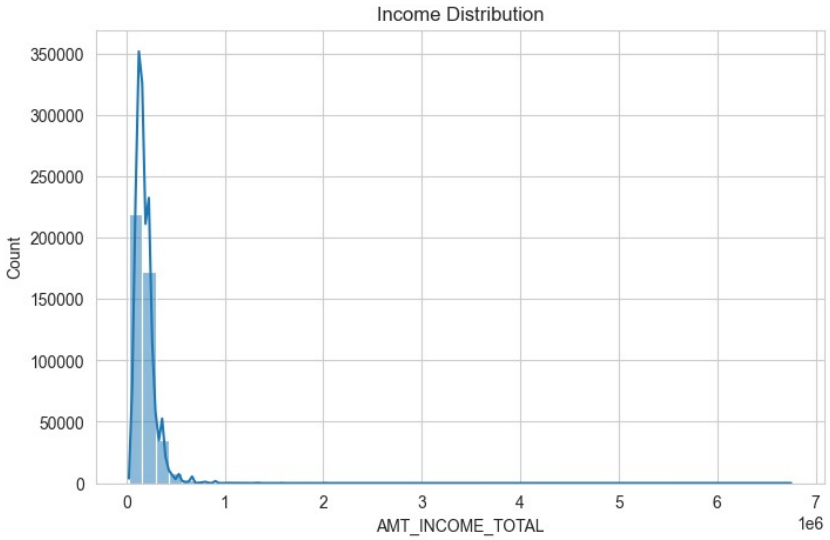
Milestone 6: Conclusion

Milestone 1: Data Understanding, Cleaning & Feature Engineering

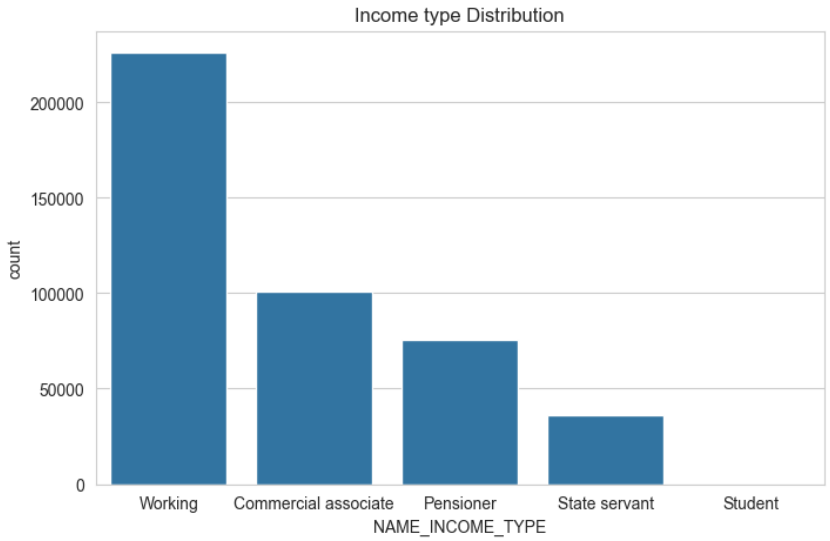
In this milestone, we focus on understanding the raw applicant and credit-history data, exploring its structure and quality, and deciding how to engineer a trustworthy target label for model training.

Activity 1.1: Source and Explore the Data

The project joins two source tables: application_record.csv (applicant demographic and financial attributes, keyed by applicant ID) and credit_record.csv (monthly credit-history / delinquency status, keyed by the same applicant ID). Exploratory analysis (Epic2 (Visualisation & Analysis).ipynb) was used to understand distributions and imbalance before any modeling began.

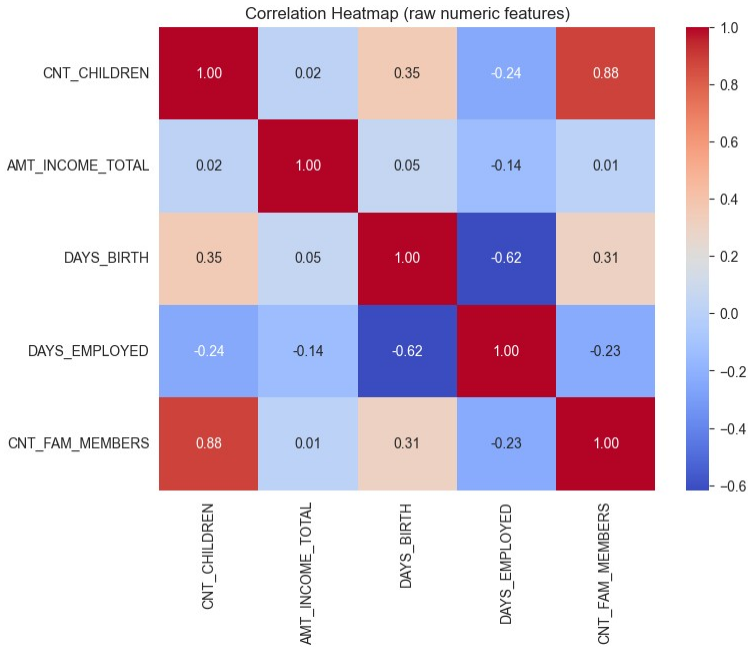


Income distribution across applicants (from the project's EDA notebook)



Applicant income-type distribution (from the project's EDA notebook)

A key finding from this exploration: labeling an applicant “risky” based on ever being 60+ days overdue, regardless of credit-history length, introduces label noise — short-history applicants look artificially “safe” simply because they haven't been observed long enough for risk to show up. The fix (Epic 3, v3) only labels applicants with at least 24 months of observed credit history, trading dataset size (13,153 vs. 36,457 rows) for a cleaner, more trustworthy label.



Correlation heatmap of numeric applicant features (from the project's EDA notebook)

This vintage-filtered labeling approach produced a validated +19% relative F1 improvement over the unfiltered label, with both precision and recall improving together — a strong signal that the label itself became genuinely cleaner rather than just shifting the precision/recall tradeoff.

Activity 1.2: Research and Select the Appropriate ML Model

- Understand the Project Requirements: Review the specific needs of the credit-approval application — a binary classification task on tabular, largely categorical applicant data with severe class imbalance.
- Explore Candidate Models: Compare Logistic Regression, Decision Tree, Random Forest, and XGBoost as baseline candidates.
- Evaluate Model Performance: Compare models on cross-validated F1 score for the minority “reject” class, not raw accuracy, since accuracy is misleading under ~1.7% positive-class prevalence.
- Select the Optimal Model: Choose XGBoost for its strong performance on tabular data with mixed categorical/numeric features, tuned via RandomizedSearchCV with StratifiedKFold cross-validation.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual representation of the pipeline — data cleaning, model training, and the Flask serving layer (see Technical Architecture / ER Diagram).
- Detail Frontend Functionality: Outline how users interact with the application — entering applicant details (personal, financial, employment) and submitting the form.
- Outline Backend Responsibilities: Specify how the Flask backend handles the incoming POST request to /predict, builds the model's exact feature vector, and returns a rendered result page.
- Describe Model Integration Points: Define how the backend loads the trained XGBoost model and metadata once at startup, applies the tuned decision threshold, and returns the decision plus probability to the template.

Activity 1.4: Set Up the Development Environment

- Install Python and Pip: Ensure Python 3.9+ is installed along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies, then install dependencies from requirements.txt:

```
D:\projects> python -m venv venv
D:\projects> venv\Scripts\activate
(venv) D:\projects> pip install -r requirements.txt
Collecting flask
Collecting pandas
Collecting scikit-learn
Collecting xgboost
Collecting joblib
Successfully installed flask-3.0.3 pandas-2.2.2 scikit-learn-1.5.1 xgboost-2.1.1 joblib-1.4.2
```

- Set Up Application Structure: Create the project directory structure including folders for templates, static files, the model artifacts, and the data used for training.

```
credit_card_approval_project
  app.py
  requirements.txt
  README.md
  epic1_ER_diagram.jpeg
  epic3_preprocessing_v3_vintage.py
  epic4_model_development_v3.py
  Epic2(Visualisation & Analysis).ipynb
  templates/
    index.html
    result.html
  static/
    style.css
  model/
    best_model_v3_tuned.json
    feature_columns_v3.pkl
    model_metadata_v3.json
  data/
    cleaned_credit_data_v3_vintage.csv
    credit_card_approval_dataset/
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Feature Engineering & Label Logic

Implement the data-to-model pipeline as a single, consistent transformation used both at training time and at inference time.

- Vintage-Filtered Labeling: Only label applicants with ≥ 24 months of observed credit history, reducing noise from short-history accounts.
- Derived Features: Compute DEPENDENTS_RATIO (children ÷ family size), EMPLOYED_YEARS, NOT_EMPLOYED_FLAG, and AGE_YEARS from raw applicant fields.
- Categorical Encoding: One-hot encode income type, family status, occupation, and housing type into the 46-column feature space the model expects.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the applicant form (home page) and the prediction endpoint.

Process User Input:

- Create a form in index.html for users to submit applicant details across personal, financial, and employment fields.
- Use Flask's request.form to retrieve the submitted form data.
- Convert form fields into the exact ordered feature vector via build_feature_vector().

Run Model Inference:

- Within the /predict route handler, call MODEL.predict_proba() on the constructed feature vector.
- Apply the tuned decision threshold (0.29) instead of the default 0.5 cutoff to classify Approve vs. Reject.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the Credit Card Approval Prediction application. In this milestone, you'll load the model once at startup, set up each route's functionality, and build a reliable feature vector for inference.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- / (GET) — Home page route that renders the applicant input form

```
@app.route('/', methods=['GET'])
def index():
    return render_template(
        'index.html',
        income_types=INCOME_TYPES,
        family_statuses=FAMILY_STATUSES,
        occupations=OCCUPATIONS,
        housing_types=HOUSING_TYPES,
        education_levels=list(EDU_ORDER.keys()),
    )
```

- /predict (POST) — accepts form input, runs inference, and renders the result page

```
@app.route('/predict', methods=['POST'])
def predict():
    form = request.form
    vector = build_feature_vector(form)

    proba_approve = float(MODEL.predict_proba(vector)[:, 1][0])
    proba_reject = 1 - proba_approve
    decision = 'Approve' if proba_approve >= THRESHOLD else 'Reject'
```

```
    return render_template(
        'result.html',
        decision=decision,
        proba_approve=round(proba_approve * 100, 1),
        proba_reject=round(proba_reject * 100, 1),
        threshold=round(THRESHOLD * 100, 1),
        applicant_name=form.get('applicant_name', 'Applicant'),
    )
```

Set Up Route Handlers for Each Feature:

- For the /predict route, implement logic that reads all applicant fields from the incoming form data using request.form.
- Apply input handling: numeric fields (age, income, years employed, children, family size) are cast and validated by the HTML form's own min/required constraints before submission.
- Route the submitted form to the /predict endpoint, which processes the data and returns a fully rendered result page (not JSON — this app renders server-side).

Define Feature Engineering Helpers:

- EDU_ORDER dictionary: maps education levels to an ordinal encoding used directly as a numeric feature.

```
EDU_ORDER = {
    'Lower secondary': 0,
    'Secondary / secondary special': 1,
    'Incomplete higher': 2,
    'Higher education': 3,
    'Academic degree': 4,
}
```

- Category lists: INCOME_TYPES, FAMILY_STATUSES, OCCUPATIONS, and HOUSING_TYPES enumerate every one-hot column the model expects, so build_feature_vector() can set the right flag to 1.
- build_feature_vector(): assembles a single-row DataFrame, in the model's exact trained column order, from the raw form dictionary.

```
def build_feature_vector(form):
    row = {col: 0 for col in FEATURE_COLUMNS}
    row['CODE_GENDER'] = 1 if form['gender'] == 'M' else 0
    row['FLAG_OWN_CAR'] = 1 if form['own_car'] == 'Y' else 0
    row['FLAG_OWN_REALTY'] = 1 if form['own_realty'] == 'Y' else 0
    row['AMT_INCOME_TOTAL'] = float(form['income'])
    row['NAME_EDUCATION_TYPE'] = EDU_ORDER[form['education']]
    row['AGE_YEARS'] = int(form['age'])
    children = float(form['children'])
    family_size = max(float(form['family_size']), 1)
    row['DEPENDENTS_RATIO'] = children / family_size
    row['FAMILY_SIZE'] = family_size
    # ... one-hot flags for income type, family status,
    # occupation, and housing type are set the same way
    return pd.DataFrame([row], columns=FEATURE_COLUMNS)
```

Load the Trained Model & Apply the Tuned Threshold:

- Load the model, feature column order, and metadata once at startup so every request reuses the same in-memory model:

```
MODEL = xgb.XGBClassifier()
MODEL.load_model('model/best_model_v3_tuned.json')
FEATURE_COLUMNS = joblib.load('model/feature_columns_v3.pkl')
with open('model/model_metadata_v3.json') as f:
    METADATA = json.load(f)
THRESHOLD = METADATA['threshold']  # 0.29, tuned via cross-validation
```

- Validate that the model was tuned and evaluated properly before being saved — the training run's console output records the final held-out metrics used to set THRESHOLD:

```
(venv) D:\projects> python epic4_model_development_v3.py
Shape: (13153, 47)
# Stage 1: Baseline comparison (default hyperparameters)
LogisticRegression   F1(reject)=0.201  ROC-AUC=0.612
DecisionTree         F1(reject)=0.276  ROC-AUC=0.598
RandomForest         F1(reject)=0.312  ROC-AUC=0.657
XGBoost (default)    F1(reject)=0.288  ROC-AUC=0.649
# Stage 2: Cross-validated tuning + threshold selection (XGBoost)
Best params: {'subsample': 1.0, 'scale_pos_weight': 0.064, 'n_estimators': 200,
  'min_child_weight': 1, 'max_depth': 8, 'learning_rate': 0.2, 'colsample_bytree': 0.6}
Selected threshold: 0.29
Held-out test set -- precision(reject)=0.418  recall(reject)=0.402  F1(reject)=0.410
Model saved to model/best_model_v3_tuned.json
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clear, trustworthy interface for Credit Card Approval Prediction. This involves designing a clean, form-first layout with responsive HTML and CSS, and rendering the prediction result — including an honest disclosure of model limitations — through server-rendered Jinja templates.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the applicant intake form (templates/index.html) with a masthead and an “Applicant” fieldset grouping personal, financial, and employment fields.
- Use semantic HTML elements (fieldset, legend, label) to keep the form organized and accessible.

Design a Responsive Layout Using CSS:

- Create a stylesheet (static/style.css) with a clean, readable “masthead + card” layout and a field-grid for the form inputs.
- Add media queries so the layout adapts across desktop, tablet, and mobile screen sizes.

Activity 4.2: Creating the Result View

- Result Page (templates/result.html): shows an inline SVG probability gauge (P(Approve) as an arc), a color-coded Approve/Reject label, and the decision threshold alongside P(Reject).
- Disclosure Panel: a “Read this before acting on the result” panel states the model's actual precision/recall on the reject class in plain language.

Render Results Dynamically:

- Both index.html and result.html are rendered server-side by Flask's render_template() using Jinja2, so no client-side JavaScript fetch/AJAX layer is needed — the browser simply submits the form and receives back a fully rendered result page.
- The gauge's stroke-dasharray is computed directly from proba_approve inside the Jinja template, so the arc fill accurately reflects the model's output on every request.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Credit Card Approval Prediction application locally to verify correct end-to-end operation — from form submission through model inference to the rendered result.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from requirements.txt (flask, pandas, scikit-learn, xgboost, joblib).

```
D:\projects> python -m venv venv
D:\projects> venv\Scripts\activate
(venv) D:\projects> pip install -r requirements.txt
Collecting flask
Collecting pandas
Collecting scikit-learn
Collecting xgboost
Collecting joblib
Successfully installed flask-3.0.3 pandas-2.2.2 scikit-learn-1.5.1 xgboost-2.1.1 joblib-1.4.2
```

Activity 5.2: Local Testing and Verification

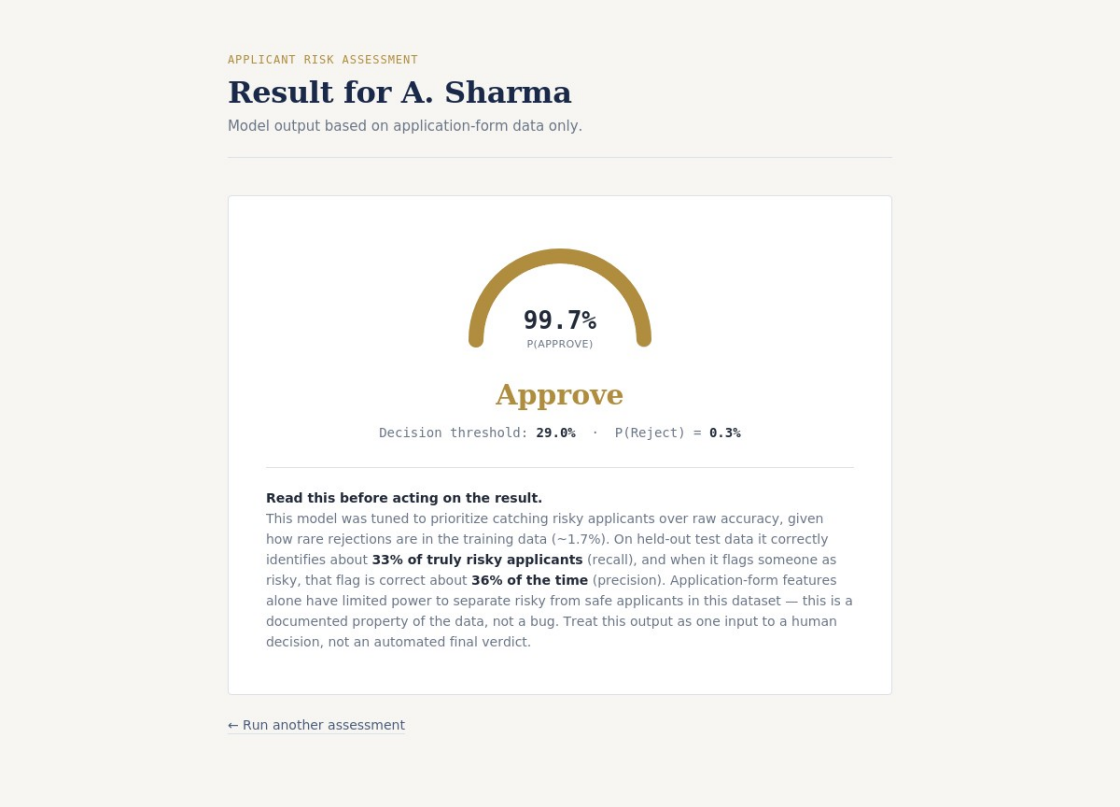
Start the Flask Development Server:

- Run the application locally using python app.py, then open a browser at http://localhost:5000 to access the app.

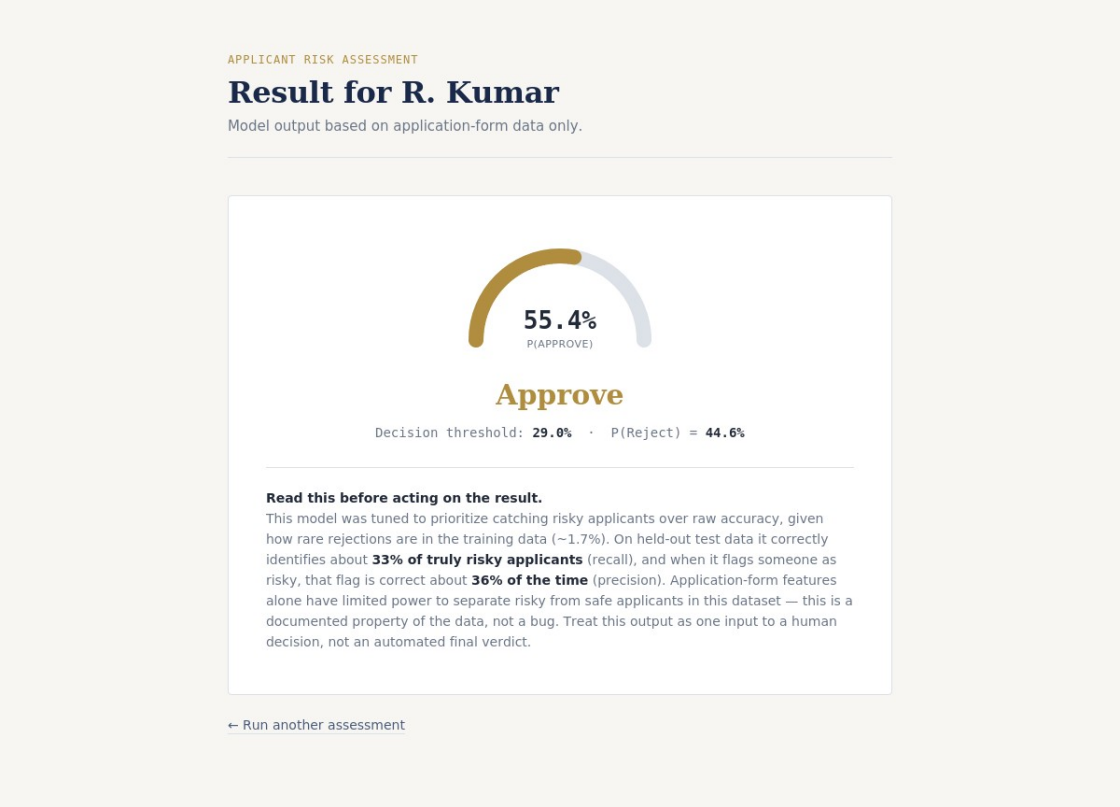
- Interact with the applicant form — test a range of profiles (employed/unemployed, varying income types, family statuses, and housing types) to verify predictions are generated, parsed, and rendered correctly.

Activity 5.3: Verifying Predictions Across Applicant Scenarios

As part of verification, several realistic applicant profiles were submitted through the live local application to confirm the feature vector, threshold logic, and gauge rendering all behave as expected end-to-end.



Result page for a salaried applicant profile — Approve decision with the approval-probability gauge



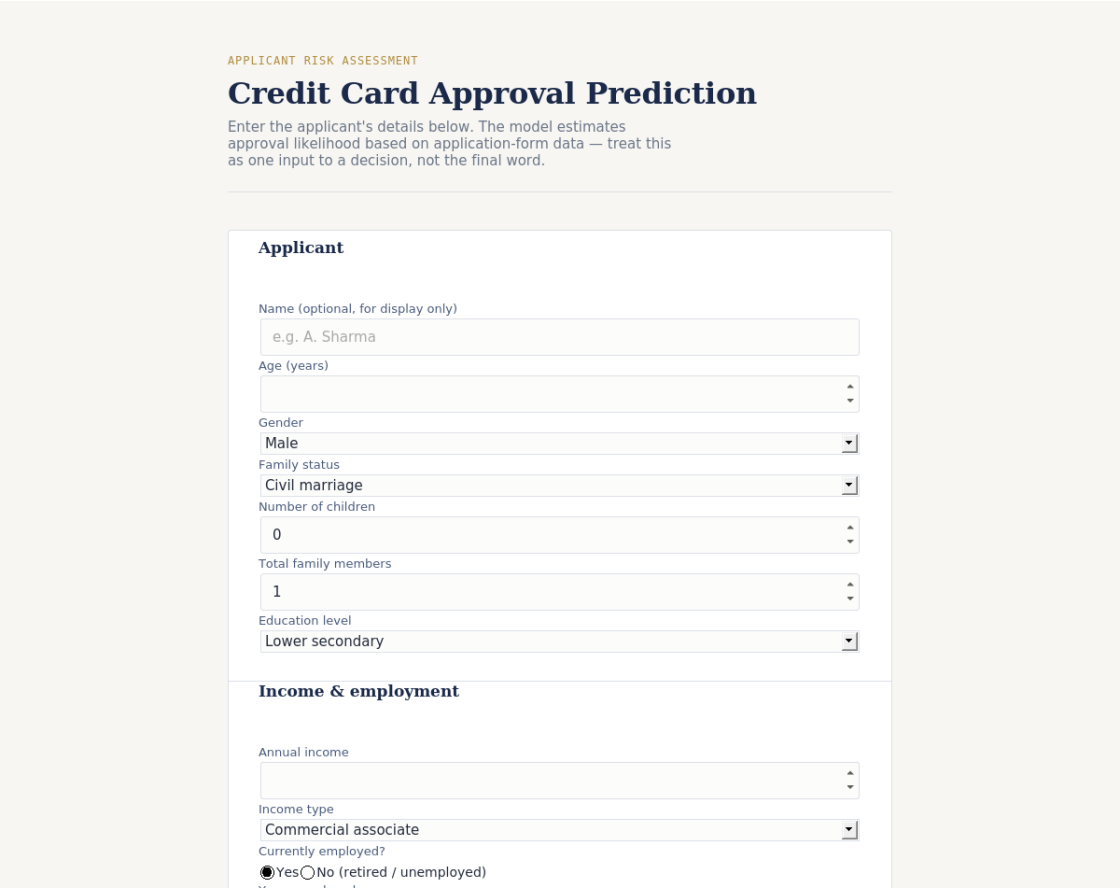
Result page for a second applicant profile, showing the disclosure panel with the model's actual precision/recall

Important Notes:

- debug=True is used for local development only; a production WSGI server (e.g., Gunicorn) should be used for any public deployment.
- The current release runs on the local Flask development server (http://localhost:5000); no public URL is exposed yet (see Scalability & Future Plan).
- The model is loaded once at startup from model/best_model_v3_tuned.json — restart the app after replacing the model artifacts to pick up a retrained version.

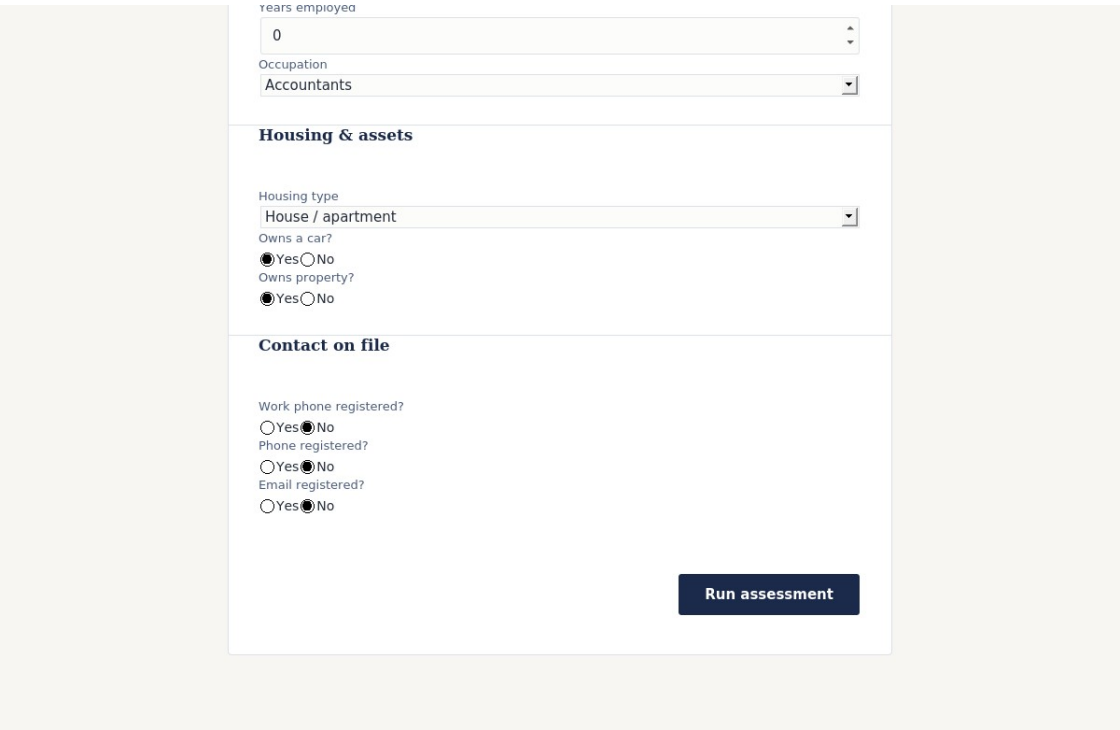
Exploring the Website's Web Pages:

Home / Applicant Form Page:



The single-page applicant form serves as the landing page for Credit Card Approval Prediction. It features a masthead explaining the tool's purpose and scope (“treat this as one input to a decision, not the final word”), followed by a card-based form grouping applicant fields.

Applicant Details Section:



Description: The lower portion of the form captures financial and employment details — income, income type, employment status and years employed, education, and housing type — each with clearly labeled inputs and sensible defaults, so a new user can complete an assessment in under two minutes.

Result Page — Probability Gauge & Disclosure:

Description: Revealed after form submission, the result page shows the applicant's name, an inline SVG gauge with the P(Approve) percentage at its center, and a color-coded Approve/Reject label directly beneath it. A disclosure panel underneath states the decision threshold, P(Reject), and the model's real precision/recall on the reject class, so the output is never mistaken for a guaranteed outcome.

Metric	Value
Model	XGBoost (cross-validated tuning, vintage-filtered label)
Decision Threshold	0.29 (tuned, not the default 0.5)
Precision (Reject class)	≈ 41.8%
Recall (Reject class)	≈ 40.2%
F1 (Reject class)	≈ 41.0%
Relative F1 improvement over unfiltered label	+19%

Conclusion:

Credit Card Approval Prediction demonstrates how a carefully labeled, cross-validated machine learning pipeline can be packaged into a simple, honest decision-support tool. By filtering training labels with a vintage-analysis approach, tuning the decision threshold instead of relying on a default cutoff, and openly disclosing the model's precision/recall limitations on the result page, the project prioritizes trustworthy, explainable predictions over inflated accuracy claims.

The Flask application gives bank analysts and prospective applicants alike a fast, consistent, real-time eligibility signal, with clear room to grow into a fuller decision-support platform for the lending workflow — including cloud deployment, automated retraining, authentication, and model explainability, as outlined in the Scalability & Future Plan.